

AWS Identity and Access Management

Users, groups, roles, policies, and the evaluation logic that ties them together — the security foundation every other AWS service depends on.

What we cover today

01 – Foundations

- Shared Responsibility Model
- What IAM is, and the root account
- Users, Groups, Roles, Policies
- Authentication vs Authorization

02 – Permissions

- Policy JSON structure
- Identity vs resource policies
- Policy evaluation logic
- Least privilege in practice
- Roles and trust policies

03 – Access & audit

- Console, CLI, SDK, access keys
- Password policy & MFA
- Credential Report, Access Advisor
- CloudTrail audit logging
- Best-practice checklist + labs

AWS Shared Responsibility Model

AWS — security of the cloud

- Infrastructure: regions, AZs, edge locations
- Physical and environmental security
- Hypervisor, host OS, network isolation
- Availability of the IAM service itself
- Compliance validation of the platform

You — security in the cloud

- Users, groups, roles and policy management
- Enabling MFA and rotating credentials
- Applying least privilege to every grant
- Reviewing permissions and audit logs
- Data encryption and application security

Every IAM decision on this deck lives on the right-hand column. AWS will never stop you from writing a bad policy.

What IAM is

- IAM is a **global service** — identities are not region-specific
- The **root account** is created with the account and should never be used for daily work or shared
- **Users** are people in your organization, and can be grouped
- **Groups** contain users only — never other groups
- A user may belong to no group, one group, or several
- IAM itself is **free**; you pay for what the identities use

ROOT ACCOUNT — LOCK IT DOWN

1. Enable MFA on the root user
2. Delete any root access keys
3. Create an admin IAM user for yourself
4. Store root credentials offline
5. Use root only for account-level tasks: closing the account, changing billing plans, certain support actions

The four building blocks

User

One physical person. Has a console password and/or access keys. Long-lived credentials.

`alice@company`

Group

A named bucket of users used to attach permissions once instead of many times.

`Developers`

Role

An identity anything can assume for temporary credentials. No password, no keys.

`EC2-S3-ReadOnly`

Policy

A JSON document that says which actions are allowed or denied on which resources.

`s3-read.json`

Identities (user, group, role) answer *who*. Policies answer *what they may do*. Neither works without the other.

How users and groups overlap

Group: Developers

Alice

Bob

Fred

AmazonEC2FullAccess

Group: Operations

David

Edward

Fred

CloudWatchFullAccess

No group

Charles

inline policy attached directly

A user can belong to **multiple groups** — Fred gets the union of both permission sets.

Groups **cannot nest**. There is no group inside a group in IAM.

Inline policies live on one identity only. Prefer **managed policies** on groups.

Authentication vs Authorization

AUTHENTICATION

Who are you?

Proving identity to AWS. Handled by passwords, MFA codes, access keys, or temporary session tokens from STS.

Failure → 401-style sign-in error

AUTHORIZATION

What may you do?

Evaluating every request against the policies attached to that identity and to the target resource.

Failure → AccessDenied

Anatomy of a policy document

Version Policy language version — always 2012-10-17

Statement Required. One or more permission statements

Effect Allow or Deny

Action API calls allowed or denied, e.g.
s3:GetObject

Condition Optional constraints — IP, MFA, tags, time

Id Optional identifier for the whole policy

Sid Optional label for one statement

Principal Who the statement applies to (resource-based policies)

Resource ARNs the actions apply to

ARN format `arn:aws:s3:::bucket/key`

A policy, in full

read-only-monitoring.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadEC2",
      "Effect": "Allow",
      "Action": "ec2:Describe*",
      "Resource": "*"
    },
    {
      "Sid": "ReadMetrics",
      "Effect": "Allow",
      "Action": [
        "cloudwatch:ListMetrics",
        "cloudwatch:GetMetricStatistics",
        "cloudwatch:Describe*"
      ],
      "Resource": "*"
    }
  ]
}
```

Wildcards — `ec2:Describe*` matches every read API on EC2. Convenient, but audit them.

Action takes a string or an array. Same for **Resource**.

No Principal here — identity-based policies are already attached to a principal.

"Resource": "*" means every resource in the account. Narrow it to ARNs wherever you can.

Conditions: the sharp tool

Deny everything unless the caller authenticated with MFA:

```
{
  "Sid": "DenyIfNoMFA",
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {
    "BoolIfExists": {
      "aws:MultiFactorAuthPresent": "false"
    }
  }
}
```

Allow S3 writes only from the office network:

```
{
  "Effect": "Allow",
  "Action": "s3:PutObject",
  "Resource": "arn:aws:s3:::reports/*",
  "Condition": {
    "IpAddress": {
      "aws:SourceIp": "203.0.113.0/24"
    }
  }
}
```

Common keys: `aws:SourceIp`, `aws:RequestedRegion`,
`aws:PrincipalTag`, `aws:CurrentTime`.

Where a policy can be attached

Identity-based

Attached to a user, group or role. Says what *that identity* can do. AWS managed, customer managed, or inline.

no Principal field

Resource-based

Attached to the resource — S3 bucket policy, KMS key policy, SQS queue policy. Enables cross-account access without a role.

Principal required

Permissions boundary

A ceiling on a user or role. Effective permissions are the intersection of the boundary and the identity policy.

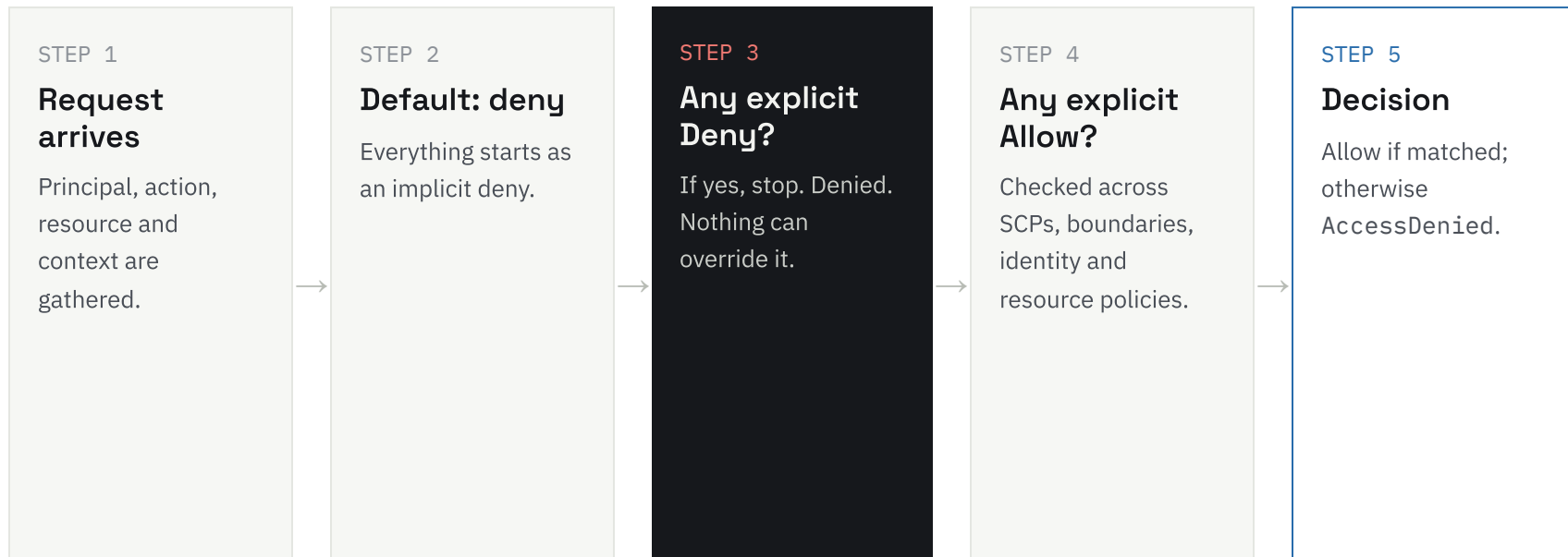
grants nothing by itself

SCP (Organizations)

Account-wide guardrail applied to an OU or account. Filters what any identity in that account can ever be granted — root included.

org-level ceiling

How AWS decides: policy evaluation



Rule of thumb: **explicit Deny** > **explicit Allow** > **implicit Deny**.

Principle of least privilege

Grant exactly the permissions needed to do the job — no more. Start from nothing and add, rather than starting from admin and trimming.

TOO BROAD

```
{
  "Effect": "Allow",
  "Action": "s3:*",
  "Resource": "*"
}
```

Every bucket, every object, including delete.

SCOPED

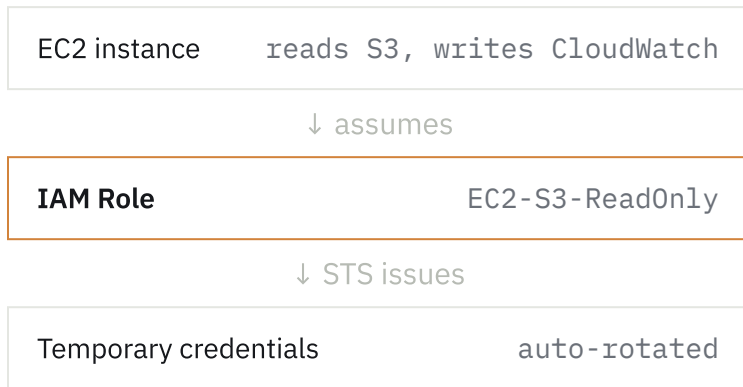
```
{
  "Effect": "Allow",
  "Action": [
    "s3:GetObject",
    "s3:PutObject"
  ],
  "Resource":
    "arn:aws:s3:::app-uploads/incoming/*"
}
```

One prefix, two actions, no delete.

ROLES

IAM Roles for AWS services

Some AWS services need to act on your behalf. You give them permissions with a role instead of embedding credentials.



- Common roles: EC2 instance roles, Lambda execution roles, CloudFormation service roles
- No long-lived keys to leak or rotate

trust-policy.json – who may assume the role

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "Service": "ec2.amazonaws.com"
    },
    "Action": "sts:AssumeRole"
  }]
}
```

```
# create the role and attach permissions
aws iam create-role --role-name EC2-S3-ReadOnly \
  --assume-role-policy-document file://trust-policy.json

aws iam attach-role-policy --role-name EC2-S3-ReadOnly \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
```

Three ways to reach AWS

01

Management Console

Browser UI. Protected by password + MFA. Best for learning, one-off tasks and inspection.

password + MFA

02

AWS CLI

Shell access to the public AWS APIs. Open source, scriptable, the alternative to clicking.

access keys

03

AWS SDK

Language libraries embedded in your application: Python, JavaScript, Java, Go, .NET, mobile, IoT. The CLI is itself built on the Python SDK.

access keys

Access keys and the CLI

Access Key ID $\approx$ username

AKIAASK4E37PV4983D6C

Secret Access Key $\approx$ password

AZPN3zojWozWCndIjhB0Unh8239a1bzbz05fqkZq

Example values — not real credentials.

- Generated in the console; users manage their own
- Shown once — the secret cannot be retrieved again
- Never commit to git, never paste in chat, never share
- Rotate on a schedule and delete unused keys

```
# configure a named profile
$ aws configure --profile dev
AWS Access Key ID      [None]: AKIA...
AWS Secret Access Key  [None]: ****
Default region name    [None]: ap-south-1
Default output format  [None]: json

# who am I right now?
$ aws sts get-caller-identity --profile dev
{
  "UserId": "AIDA...",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/alice"
}

# rotate: create new, test, then delete old
$ aws iam create-access-key --user-name alice
$ aws iam delete-access-key --user-name alice \
    --access-key-id AKIA_OLD_KEY_ID
```

Account password policy

- Set a minimum password length
- Require character types: uppercase, lowercase, numbers, symbols
- Allow users to change their own passwords
- Expire passwords after a set number of days
- Prevent password re-use
- Require an admin reset after expiry, if you want a hard stop

Set it once, for the whole account

```
aws iam update-account-password-policy \  
  --minimum-password-length 14 \  
  --require-uppercase-characters \  
  --require-lowercase-characters \  
  --require-numbers \  
  --require-symbols \  
  --allow-users-to-change-password \  
  --max-password-age 90 \  
  --password-reuse-prevention 5
```

Multi-Factor Authentication

Password *you know* + Security device *you own* = **A stolen password is not enough**

Virtual MFA device

Google Authenticator, Authy. Authy supports multiple tokens on one device.

U2F security key

YubiKey by Yubico. One key can cover multiple root and IAM users.

Hardware key fob

Physical TOTP fob provided by Gemalto.

GovCloud key fob

Fob for AWS GovCloud (US), provided by SurePassID.

Enable MFA on the root account first. Then every user with administrative permissions — and enforce it with a policy condition.

Knowing what is actually happening

IAM Credentials Report

account level

CSV listing every user and the state of their credentials: password age, MFA status, key age, last used.

IAM Access Advisor

user level

Shows which services a user was granted and when each was last accessed. Use it to strip permissions nobody uses.

AWS CloudTrail

API history

Records every API call — who, when, from where, with what result. The record you reach for after an incident.

```
# generate and download the credential report
aws iam generate-credential-report
aws iam get-credential-report --query Content --output text | base64 -d > report.csv

# what has this user actually used?
aws iam generate-service-last-accessed-details --arn arn:aws:iam::123456789012:user/alice

# create a trail that logs to S3 in all regions
aws cloudtrail create-trail --name org-trail --s3-bucket-name my-audit-logs --is-multi-region-trail
aws cloudtrail start-logging --name org-trail
```

PRACTICE

IAM guidelines & best practices

01 Use the root account only for account setup

03 Assign users to groups; attach permissions to groups

05 Require MFA everywhere, starting with root

07 Use access keys only for programmatic access, and rotate them

09 Audit with Credentials Report, Access Advisor and CloudTrail

02 One physical person = one IAM user

04 Create and enforce a strong password policy

06 Use roles for services instead of embedded keys

08 Apply least privilege to every grant you write

10 Never share IAM users or access keys

Hands-on labs

Lab 1.3 Set up billing alerts and budgets

Lab 2.1 Secure the root account with MFA

Lab 2.3 Create custom IAM policies — JSON practice

Lab 2.5 Enable CloudTrail for audit logging

Lab 1.5 AWS Pricing Calculator — estimate a sample workload

Lab 2.2 Create IAM users and groups with different permissions

Lab 2.4 Create and assume an IAM role for EC2

```
# Lab 2.2 in four commands
aws iam create-group --group-name Developers
aws iam create-user --user-name alice
aws iam add-user-to-group --user-name alice --group-name Developers
aws iam attach-group-policy --group-name Developers \
    --policy-arn arn:aws:iam::aws:policy/AmazonEC2ReadOnlyAccess
```

WRAP-UP

IAM in one slide

Users	One physical person, with a console password
--------------	--

Policies	JSON documents that grant or deny actions
-----------------	---

Evaluation	Explicit Deny > explicit Allow > implicit Deny
-------------------	--

CLI / SDK	Command line and language libraries, on access keys
------------------	---

Groups	Contain users only, never other groups
---------------	--

Roles	Temporary credentials for services and cross-account access
--------------	---

Security	MFA plus a password policy
-----------------	----------------------------

Audit	Credentials Report, Access Advisor, CloudTrail
--------------	--